DAA LAB

1.LINEAR SEARCH AND TIME ANALYSIS

Aim: To write a C program to implement Linear Search, determine the time required to search for an element, repeat the experiment for different values of n, and plot a graph of the time taken versus n using a graphical library.

Simplified Code:

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <graphics.h>

int main() {
    int n_list[] = {10000, 20000, 30000, 40000, 50000};
    double results[5];
    int gd = DETECT, gm;
    int i, k, x = -1;

    for (k = 0; k < 5; k++) {
        int n = n_list[k];
        int *arr = (int *)malloc(n * sizeof(int));
        for (i = 0; i < n; i++) arr[i] = i;

        clock_t start = clock();
        for (i = 0; i < n; i++) {
            if (arr[i] == x) break;
        }
        clock_t end = clock();

        results[k] = ((double)(end - start)) / CLOCKS_PER_SEC;
        free(arr);
    }

    initgraph(&gd, &gm, "");
    line(50, 400, 500, 400);
    line(50, 400, 50, 50);
    outtextxy(200, 420, "Number of Elements (n)");
    outtextxy(10, 100, "Time");
```

```
    for (k = 0; k < 5; k++) {
        int px = 50 + (k + 1) * 70;
        int py = 400 - (int)(results[k] * 10000000);
        circle(px, py, 3);
        if (k > 0) {
            int prevX = 50 + k * 70;
            int prevY = 400 - (int)(results[k-1] * 10000000);
            line(prevX, prevY, px, py);
        }
    }

    getch();
    closegraph();
    return 0;
}
```

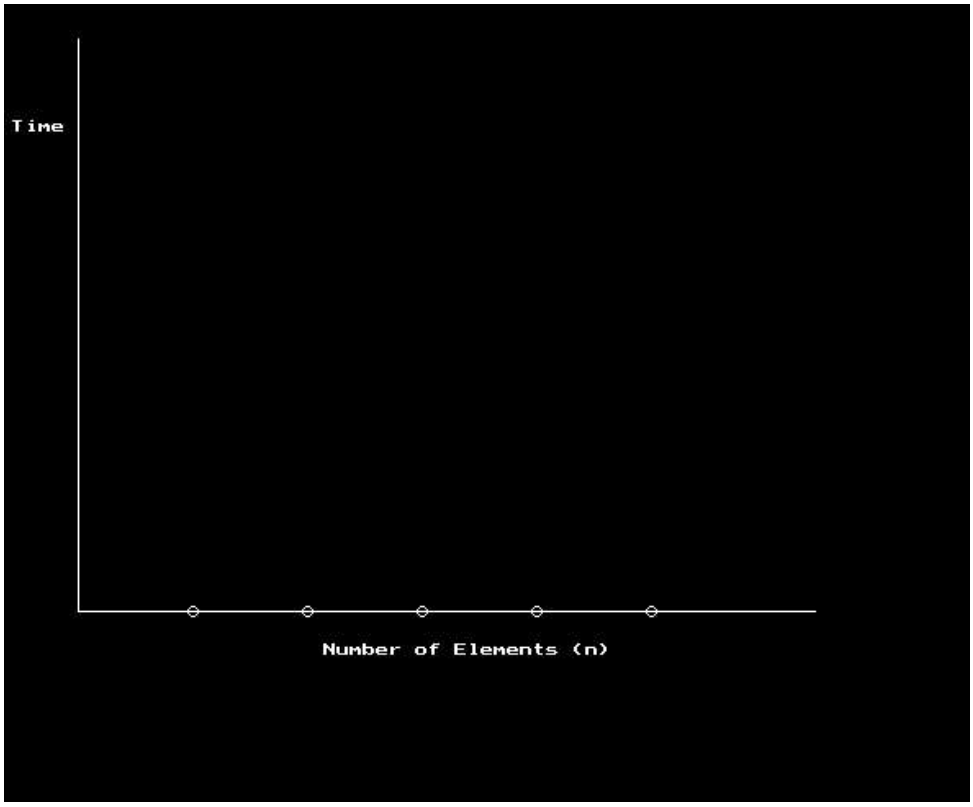
Algorithm:

- **Step 1:** Set i to 1.
- **Step 2:** If $i > n$ then go to step 7.
- **Step 3:** If $A[i] = x$ then go to step 6.
- **Step 4:** Set i to $i + 1$.
- **Step 5:** Go to Step 2.
- **Step 6:** Printf Element x Found at index i and go to step 8.
- **Step 7:** Printf element not found.
- **Step 8:** Exit.

Child-Level Algorithm:

- **Step 1:** Line up a row of boxes and grab a stopwatch.
- **Step 2:** Start the timer and open the first box.
- **Step 3:** If you find your toy, stop the timer immediately.
- **Step 4:** If the toy isn't there, move to the very next box.
- **Step 5:** Repeat until you find the toy or check every box.
- **Step 6:** Draw a dot on a chart: the more boxes you check, the higher the dot goes because the timer took longer.
- **Step 7:** Connect your dots to see a straight line pointing up.

Output:



Result: Thus, the C program for linear search algorithm was executed successfully and the output was verified.

2.RECURSIVE BINARY SEARCH AND TIME ANALYSIS

Aim: To write a C program to implement recursive Binary Search, determine the time required to search for an element, repeat the experiment for different values of n, and plot a graph of the time taken versus n.

Simplified Code:

C

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <graphics.h>

int binarySearch(int arr[], int low, int high, int x) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == x) return mid;
        if (arr[mid] > x) return binarySearch(arr, low, mid - 1, x);
        return binarySearch(arr, mid + 1, high, x);
    }
    return -1;
}

int main() {
    int n_list[] = {10000, 20000, 30000, 40000, 50000};
    double results[5];
    int gd = DETECT, gm, i, k;

    for (k = 0; k < 5; k++) {
        int n = n_list[k];
```

```
    int *arr = (int *)malloc(n * sizeof(int));
    for (i = 0; i < n; i++) arr[i] = i;

    clock_t start = clock();
    binarySearch(arr, 0, n - 1, -1);
    clock_t end = clock();

    results[k] = ((double)(end - start)) / CLOCKS_PER_SEC;
    free(arr);
}

initgraph(&gd, &gm, "");
line(50, 400, 500, 400);
line(50, 400, 50, 50);
outtextxy(200, 420, "Number of Elements (n)");
outtextxy(10, 80, "Time");

for (k = 0; k < 5; k++) {
    int px = 50 + (k + 1) * 70;
    int py = 400 - (int)(results[k] * 100000000);
    circle(px, py, 3);
    if (k > 0) {
        int prevX = 50 + k * 70;
        int prevY = 400 - (int)(results[k-1] * 100000000);
        line(prevX, prevY, px, py);
    }
}

getch();
closegraph();
```

```
    return 0;
}
```

Visual Complexity Representation:

The graph for Binary Search is **logarithmic.** Unlike the straight line of linear search, this curve flattens out quickly because the search space is cut in half at every step.

Algorithm:

- **Step 1**: Compare x with the middle element.
- **Step 2**: If x matches with the middle element, we return the mid index.
- **Step 3**: Else if x is greater than the mid element, then x can only lie in the right half subarray after the mid element.
- **Step 4**: Recur for the right half.
- **Step 5**: Else (if x is smaller) recur for the left half.

Child-Level Algorithm:

- **Step 1**: Imagine you are looking for a word in a dictionary.
- **Step 2**: Open the dictionary exactly in the middle.
- **Step 3**: If your word is there, you found it!
- **Step 4**: If your word comes *after* the middle, throw away the first half and keep the second half.
- **Step 5**: If your word comes *before* the middle, throw away the second half and keep the first half.
- **Step 6**: Repeat this "splitting in half" until you find the word.

Result: Thus, the C program for recursive binary search algorithm was executed successfully and the output was verified.

3.PATTERN SEARCH (KMP LOGIC)

Aim: To write a C program that, given a text txt [0...n-1] and a pattern pat [0...m-1], prints all occurrences of the pattern in the text. You may assume that $n > m$.

Simplified Code:

C

```c
#include <stdio.h>
#include <string.h>

void computeLPS(char* pat, int M, int* lps) {
    int len = 0, i = 1;
    lps[0] = 0;
    while (i < M) {
        if (pat[i] == pat[len]) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) len = lps[len - 1];
            else { lps[i] = 0; i++; }
        }
    }
}

void search(char* pat, char* txt) {
    int M = strlen(pat);
    int N = strlen(txt);
    int lps[M];
    computeLPS(pat, M, lps);
```

```
    int i = 0, j = 0;
    while (i < N) {
        if (pat[j] == txt[i]) { j++; i++; }
        if (j == M) {
            printf("Found pattern at index %d\n", i - j);
            j = lps[j - 1];
        } else if (i < N && pat[j] != txt[i]) {
            if (j != 0) j = lps[j - 1];
            else i++;
        }
    }
}

int main() {
    char txt[] = "ABABDABACDABABCABAB";
    char pat[] = "ABABCABAB";
    search(pat, txt);
    return 0;
}
```

Visual Representation of Pattern Matching:

Algorithm:

- **Step 1:** Start the comparison of pat[j] where $j=0$ with characters of the current window of text.
- **Step 2:** Keep matching characters txt[i] and pat[j] and increment i and j while they match.
- **Step 3:** When a mismatch occurs:
    - 1) We know pat[0...j-1] matches with txt[i-j...i-1].
    - 2) Note that j starts with 0 and increments only when there is a match.
    - 3) We use lps[j-1] (longest proper prefix which is also a suffix) to skip unnecessary comparisons.

Child-Level Algorithm:

- **Step 1:** Imagine you are looking for the word "APPLE" in a giant soup of letters.
- **Step 2:** Start at the beginning of the soup and check the first letter.
- **Step 3:** If you see an 'A', look for 'P' next. If they keep matching, keep going!
- **Step 4:** If you see a wrong letter (like "APPX"), don't start all the way back at the beginning of the soup.
- **Step 5:** Use a "cheat sheet" (the LPS array) to remember if the part you already checked looks like the start of the word you are looking for.
- **Step 6:** This helps you skip letters you've already seen so you can find the word much faster!

Output:

```
Found pattern at index 10
```

Result: Thus, the C program for Pattern search algorithm was executed successfully and the output was verified.

4.INSERTION SORT AND HEAP SORT TIME ANALYSIS

Aim: To write a C program to sort a given set of elements using Insertion sort and Heap sort methods, determine the time required to sort, repeat for different values of n, and plot a graph of time taken versus n for both methods.

Simplified Code:

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <graphics.h>

void insertionSort(int arr[], int n) {
    int i, key, j;
    for (i = 1; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

void heapify(int arr[], int n, int i) {
    int largest = i, l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && arr[l] > arr[largest]) largest = l;
    if (r < n && arr[r] > arr[largest]) largest = r;
```

```
    if (largest != i) {
        int temp = arr[i]; arr[i] = arr[largest]; arr[largest] = temp;
        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--) heapify(arr, n, i);
    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;
        heapify(arr, i, 0);
    }
}

int main() {
    int n_list[] = {5000, 10000, 15000, 20000};
    double i_times[4], h_times[4];
    int gd = DETECT, gm, k, i;

    for (k = 0; k < 4; k++) {
        int n = n_list[k];
        int *a1 = malloc(n * sizeof(int)), *a2 = malloc(n * sizeof(int));
        for (i = 0; i < n; i++) a1[i] = a2[i] = rand() % 1000;

        clock_t s1 = clock(); insertionSort(a1, n); clock_t e1 = clock();
        i_times[k] = (double)(e1 - s1) / CLOCKS_PER_SEC;

        clock_t s2 = clock(); heapSort(a2, n); clock_t e2 = clock();
        h_times[k] = (double)(e2 - s2) / CLOCKS_PER_SEC;
```

```
    free(a1); free(a2);
  }

  initgraph(&gd, &gm, "");
  line(50, 400, 500, 400); line(50, 400, 50, 50);
  outtextxy(50, 30, "Yellow: Insertion | White: Heap");

  for (k = 0; k < 4; k++) {
    int px = 50 + (k + 1) * 100;
    int py_i = 400 - (int)(i_times[k] * 1000); // Scaled for display
    int py_h = 400 - (int)(h_times[k] * 1000);

    setcolor(YELLOW); circle(px, py_i, 3);
    setcolor(WHITE); circle(px, py_h, 3);
    if (k > 0) {
      setcolor(YELLOW); line(50 + k * 100, 400 - (int)(i_times[k-1] * 1000), px, py_i);
      setcolor(WHITE); line(50 + k * 100, 400 - (int)(h_times[k-1] * 1000), px, py_h);
    }
  }
  getch(); closegraph(); return 0;
}
```

Algorithm (Exhaustive Breakdown):

- **Insertion Sort:**
  - Iterate from the second element to the last element.
  - Compare the current "key" element with its predecessor.
  - If the key is smaller, shift greater elements one position up to create space.
- **Heap Sort:**
  - Build a max heap from the input data.
  - Swap the root (maximum element) with the last item and reduce heap size.
  - Heapify the root to restore heap property and repeat until sorted.

Child-Level Algorithm:

- **Insertion Sort**: Imagine you are sorting a hand of playing cards. You take one card at a time and slide it into the correct spot among the cards you are already holding.
- **Heap Sort:** Imagine a pile of toys. First, you organize them into a "mountain" where the biggest toy is always at the very top. You take the top toy, put it in a box, and reorganize the mountain so the next biggest toy is on top. Keep doing this until the pile is empty.

Output: Graph with both Insertion sort and heap with different color

---

Result: Thus, the C program for Insertion and Heap sort algorithms was executed successfully and the output was verified.

5.GRAPH TRAVERSAL USING BREADTH FIRST SEARCH (BFS)

Aim: To write a C program to implement graph traversal using Breadth First Search (BFS).

Simplified Code:

C

```c
#include <stdio.h>
#include <stdlib.h>

void BFS(int adj[10][10], int visited[10], int n, int start) {
    int queue[10], front = 0, rear = -1, i;

    printf("%d ", start);
    visited[start] = 1;
    queue[++rear] = start;

    while (front <= rear) {
        int current = queue[front++];
        for (i = 1; i <= n; i++) {
            if (adj[current][i] == 1 && !visited[i]) {
                printf("%d ", i);
                visited[i] = 1;
                queue[++rear] = i;
            }
        }
    }
}

int main() {
```

```
    int n, i, j, adj[10][10], visited[10] = {0}, start;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &adj[i][j]);
    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("BFS Traversal: ");
    BFS(adj, visited, n, start);
    return 0;
}
```

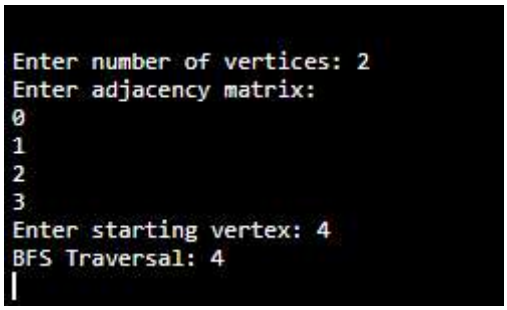
Algorithm:

- **Step 1:** Consider the graph you want to navigate.
- **Step 2:** Select any vertex in your graph (e.g., $v1$) from which you want to traverse.
- **Step 3:** Initialize a Visited array and a Queue data structure.
- **Step 4:** Add the starting vertex to the visited array and its neighbors to the queue.
- **Step 5:** Using FIFO (First In First Out), remove an element from the queue, mark it visited, and add its unvisited neighbors to the queue.
- **Step 6:** Repeat until the queue is empty.

Child-Level Algorithm:

- **Step 1:** Imagine you are standing at the center of a spider web.
- **Step 2:** First, you visit all the spots exactly one "step" away from you.
- **Step 3:** Only after you finish those, you move to the spots that are two "steps" away.
- **Step 4:** You use a "Waiting Line" (Queue) to remember who to visit next.
- **Step 5:** You never visit the same spot twice!

Output:



Result: Thus, the C program for BFS graph traversal was executed successfully and the output was verified.

6.GRAPH TRAVERSAL USING DEPTH FIRST SEARCH (DFS)

Aim: To write a C program to implement graph traversal using Depth First Search (DFS).

Simplified Code:

C

```c
#include <stdio.h>

int adj[20][20], visited[20], n;

void DFS(int v) {
    int i;
    printf("%d ", v);
    visited[v] = 1;
    for (i = 1; i <= n; i++) {
        if (adj[v][i] == 1 && !visited[i]) {
            DFS(i);
        }
    }
}

int main() {
    int i, j, start;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &adj[i][j]);
```

```
        }
    }
    printf("Enter starting vertex: ");
    scanf("%d", &start);
    printf("DFS Traversal: ");
    DFS(start);
    return 0;
}
```

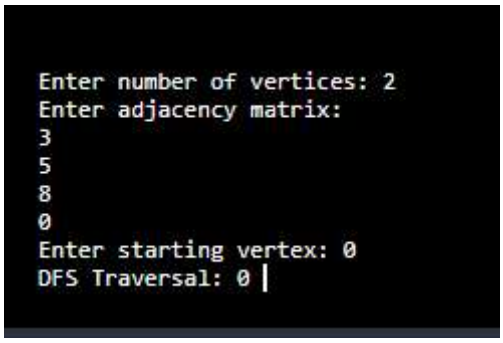
Algorithm:

- **Step 1:** Start by putting any one of the graph's vertices into the stack (or use recursion).
- **Step 2:** Take the current vertex and add it to the visited list.
- **Step 3:** Create a list of that vertex's adjacent nodes.
- **Step 4:** For each adjacent node not in the visited list, move to that node and repeat the process immediately (go deeper).
- **Step 5:** Continue until all reachable nodes are visited and the stack is empty.

Child-Level Algorithm:

- **Step 1:** Imagine you are exploring a deep cave with many branching tunnels.
- **Step 2:** Pick one tunnel and walk as far as you can until you hit a dead end.
- **Step 3:** While walking, drop breadcrumbs so you know where you have already been.
- **Step 4:** When you hit a dead end, backtrack to the last branch you saw and try a different tunnel.
- **Step 5:** Keep going deep into every new tunnel you find until you have explored the whole cave.

Output:



Result: Thus, the C program for depth first search algorithm was executed successfully and the output was verified.

7.DIJKSTRA'S SHORTEST PATH ALGORITHM

Aim: To write a C program from a given vertex in a weighted connected graph to find the shortest paths to all other vertices using Dijkstra's algorithm.

Simplified Code:

C

```c
#include <stdio.h>

#define INF 999


void dijkstra(int n, int cost[10][10], int src) {
    int dist[10], visited[10], count, min, nextnode, i, j;
    for (i = 1; i <= n; i++) {
        dist[i] = cost[src][i];
        visited[i] = 0;
    }
    dist[src] = 0;
    visited[src] = 1;
    count = 1;
    while (count < n) {
        min = INF;
        for (i = 1; i <= n; i++)
            if (dist[i] < min && !visited[i]) {
                min = dist[i];
                nextnode = i;
            }
        visited[nextnode] = 1;
        for (i = 1; i <= n; i++)
            if (!visited[i])
                if (min + cost[nextnode][i] < dist[i])
```

```
                dist[i] = min + cost[nextnode][i];
        count++;
    }
    for (i = 1; i <= n; i++)
        if (i != src) printf("\nDistance to %d: %d", i, dist[i]);
}

int main() {
    int n, cost[10][10], i, j, s;
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter weight matrix (use 999 for infinity):\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &cost[i][j]);
    printf("Enter source: ");
    scanf("%d", &s);
    dijkstra(n, cost, s);
    return 0;
}
```

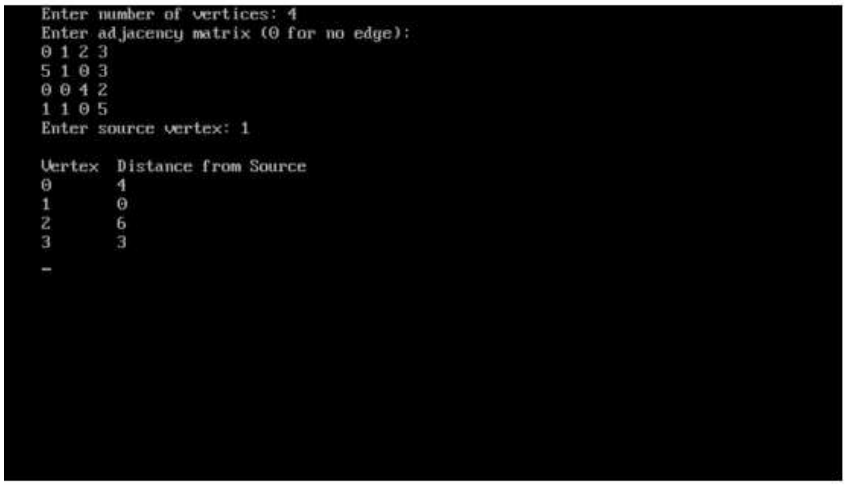
Algorithm:

- **Step 1:** Mark the source node with a current distance of 0 and all other nodes with infinity.
- **Step 2:** Select the non-visited node with the smallest current distance as the "current node".
- **Step 3:** For each neighbor of the current node, add the distance of the current node to the weight of the edge connecting them.
- **Step 4:** If this sum is smaller than the neighbor's current recorded distance, update it as the new shortest distance.
- **Step 5:** Mark the current node as visited.
- **Step 6:** Repeat steps 2-5 until all nodes are visited.

Child-Level Algorithm:

- **Step 1:** Imagine you are at your house and want to find the quickest way to all your friends' houses.
- **Step 2:** Write "0 minutes" on your front door and "Infinity" on every friend's door because you don't know the way yet.
- **Step 3:** Look at the friends you can reach directly. Write down how long it takes to walk to them.
- **Step 4:** Always pick the friend's house that is currently the closest to you.
- **Step 5:** From that house, look at their neighbors. If going through your friend's yard is a shortcut to someone else, update the time on their door!
- **Step 6:** Put a sticker on the door once you are sure you found the absolute fastest way.

Output:



Result: Thus, the C program for finding shortest paths using Dijkstra's algorithm was executed successfully and the output was verified.

8.MINIMUM COST SPANNING TREE (PRIM'S ALGORITHM)

Aim: To write a C program to find the minimum cost spanning tree of a given undirected graph using Prim's algorithm.

Simplified Code:

C

```c
#include <stdio.h>


int main() {
    int n, i, j, ne = 1;
    int visited[10] = {0}, min, a, b, u, v, mincost = 0;
    int cost[10][10];


    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (use 999 for infinity):\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            scanf("%d", &cost[i][j]);
            if (cost[i][j] == 0) cost[i][j] = 999;
        }
    }


    visited[1] = 1;
    while (ne < n) {
        min = 999;
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                if (cost[i][j] < min) {
```

```
                if (visited[i] != 0) {
                    min = cost[i][j];
                    a = u = i;
                    b = v = j;
                }
            }
        }
    }
    if (visited[u] == 0 || visited[v] == 0) {
        printf("\n Edge %d:(%d %d) cost:%d", ne++, a, b, min);
        mincost += min;
        visited[b] = 1;
    }
    cost[a][b] = cost[b][a] = 999;
  }
  printf("\n Minimum cost = %d", mincost);
  return 0;
}
```

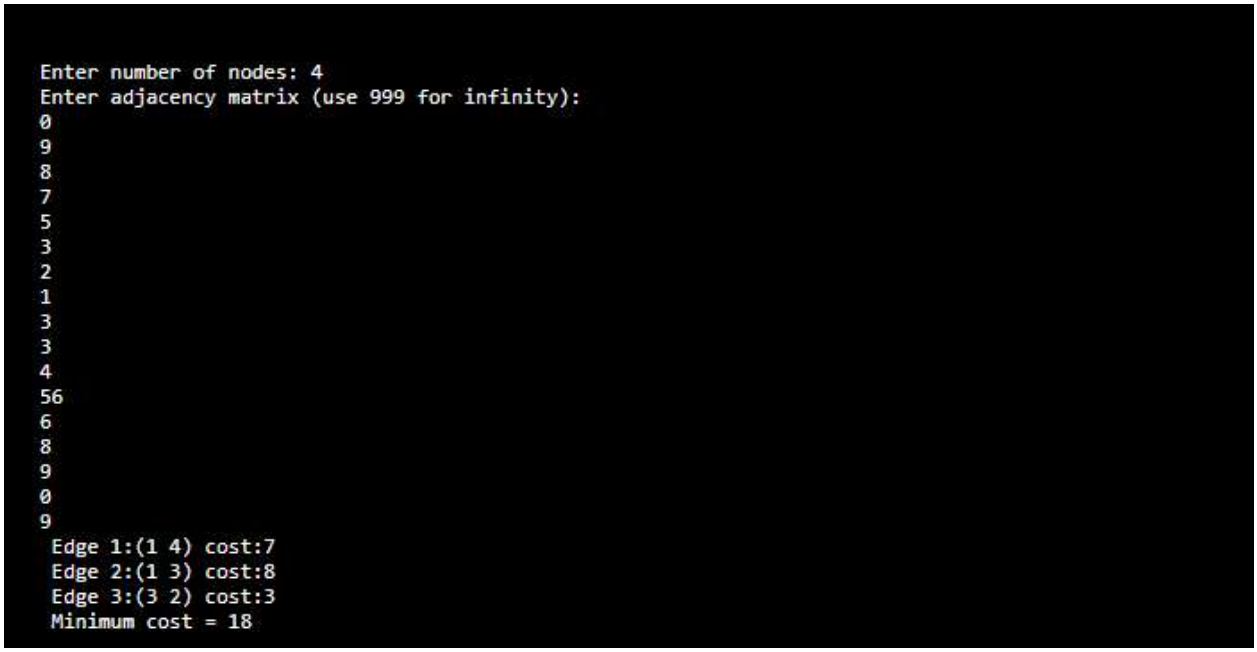
Algorithm:

- **Step 1:** Determine an arbitrary vertex as the starting vertex of the MST.
- **Step 2:** Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertices).
- **Step 3:** Find edges connecting any tree vertex with the fringe vertices.
- **Step 4:** Find the minimum among these edges.
- **Step 5:** Add the chosen edge to the MST if it does not form any cycle.
- **Step 6:** Return the MST and exit.

Child-Level Algorithm:

- **Step 1:** Imagine you want to connect several houses with electricity cables using the least amount of wire.
- **Step 2:** Start at any house and mark it as "connected."
- **Step 3:** Look at all the cables you could plug into your connected house to reach a house that is still dark.
- **Step 4:** Pick the shortest (cheapest) cable available.
- **Step 5:** Now that new house is also "connected."
- **Step 6:** Keep picking the shortest cable that reaches a dark house until everyone has power!

Output:



Result: Thus, the C program for Prim's algorithm was executed successfully and the output was verified.

9.FLOYD’S ALL-PAIRS SHORTEST PATHS ALGORITHM

Aim: To write a C program to implement Floyd’s algorithm for the All-Pairs Shortest Paths problem.

Simplified Code:

C

```c
#include <stdio.h>

#define INF 999

void floyd(int n, int dist[10][10]) {
    int i, j, k;
    for (k = 1; k <= n; k++) {
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}

int main() {
    int n, i, j, dist[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter weight matrix (use 999 for INF):\n");
    for (i = 1; i <= n; i++) {
```

```
        for (j = 1; j <= n; j++) {
            scanf("%d", &dist[i][j]);
        }
    }
    floyd(n, dist);
    printf("\nAll-Pairs Shortest Paths Matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (dist[i][j] == INF) printf("INF\t");
            else printf("%d\t", dist[i][j]);
        }
        printf("\n");
    }
    return 0;
}
```

Algorithm:

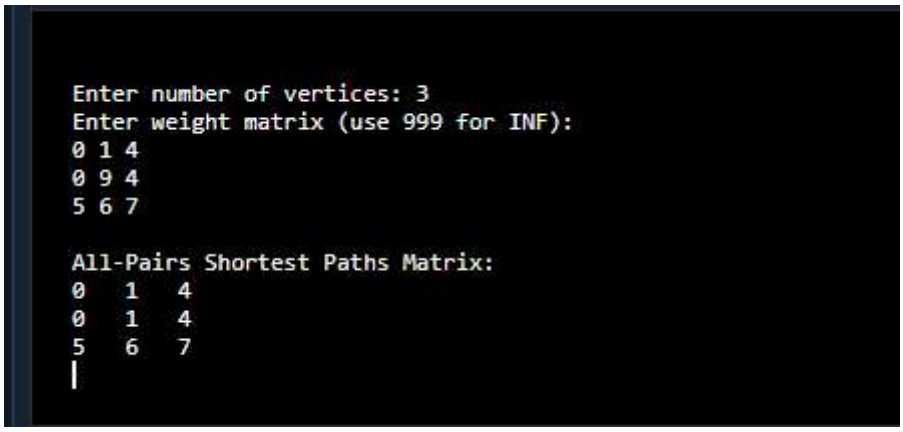
- **Step 1:** Initialize the solution matrix to be the same as the input graph matrix.
- **Step 2:** Update the solution matrix by considering every vertex, one by one, as an intermediate vertex.
- **Step 3:** When considering vertex k as an intermediate point, you have already considered vertices 0 to $k-1$.
- **Step 4:** For every pair (i, j), there are two cases:
    - Case A: k is not an intermediate vertex; keep the current distance `dist[i][j]`.
    - Case B: k is an intermediate vertex; update `dist[i][j]` if the path through k (`dist[i][k] + dist[k][j]`) is shorter than the current `dist[i][j]`.

Child-Level Algorithm:

- **Step 1:** Imagine you are making a map of travel times between every single house in your neighborhood.
- **Step 2:** Start with a chart showing the direct paths you already know.

- **Step 3:** Now, pick one house to be a "Pit Stop" (Intermediate vertex).
- **Step 4:** Check every pair of houses. Ask yourself: "Is it faster to go directly from House A to House B, or is it faster to stop at the Pit Stop house along the way?"
- **Step 5:** If the Pit Stop saves time, update your chart with the new, faster time.
- **Step 6:** Repeat this for every house in the neighborhood until your chart shows the absolute fastest way between any two points.

Output:



Result: Thus, the C program for Floyd's algorithm was executed successfully and the output was verified.

10.TRANSITIVE CLOSURE (WARSHALL'S ALGORITHM)

Aim: To write a C program to compute the transitive closure of a given directed graph using Warshall's algorithm.

Simplified Code:

C

```c
#include <stdio.h>

void warshall(int n, int adj[10][10]) {
    int i, j, k;
    for (k = 1; k <= n; k++) {
        for (i = 1; i <= n; i++) {
            for (j = 1; j <= n; j++) {
                adj[i][j] = adj[i][j] || (adj[i][k] && adj[i][k]);
            }
        }
    }
}

int main() {
    int n, i, j, adj[10][10];
    printf("Enter number of vertices: ");
    scanf("%d", &n);
    printf("Enter adjacency matrix (0/1):\n");
    for (i = 1; i <= n; i++)
        for (j = 1; j <= n; j++)
            scanf("%d", &adj[i][j]);

    warshall(n, adj);
```

```
    printf("\nTransitive Closure Matrix:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", adj[i][j]);
        printf("\n");
    }
    return 0;
}
```

Algorithm:

- **Step 1:** Start the program.
- **Step 2:** Create a Boolean reachability matrix where a value of 1 means a path exists between two nodes and 0 means it does not.
- **Step 3:** Use logical operations (OR and AND) to update the matrix.
- **Step 4:** Check every pair of nodes (i, j) and see if they can be connected through an intermediate node k.
- **Step 5:** Stop the program.

Child-Level Algorithm:

- **Step 1:** Imagine you are looking at a map of one-way streets between treehouses.
- **Step 2:** Right now, your map only shows "Direct Jumps" from one treehouse to the next.
- **Step 3:** You want to update the map to show if you can get from Treehouse A to Treehouse B even if you have to go through other treehouses first.
- **Step 4:** Pick one treehouse to be a "Middle Hub."
- **Step 5:** Look at every pair of treehouses. Ask: "Can I get from A to B by stopping at the Middle Hub?"
- **Step 6:** If the answer is "Yes," draw a line on your map between A and B.
- **Step 7:** Do this for every treehouse until your map shows all possible connections!

Output:

```
Enter number of vertices: 4
Enter adjacency matrix (0/1):
0 0 0 0
1 0 0 0
0 1 0 0
0 0 1 0

Transitive Closure Matrix:
0 0 0 0
1 1 1 1
1 1 1 1
1 1 1 1


=== Code Execution Successful ===
```

Result: Thus, the C program for Warshall's algorithm was executed successfully and the output was verified.

11.MAXIMUM AND MINIMUM NUMBERS (DIVIDE AND CONQUER)

Aim: To write a C program to find out the maximum and minimum numbers in a given list of n numbers using the divide and conquer technique.

Simplified Code:

C

```c
#include <stdio.h>

struct pair {
    int min;
    int max;
};

struct pair getMinMax(int arr[], int low, int high) {
    struct pair minmax, left, right;
    int mid;

    if (low == high) {
        minmax.max = arr[low];
        minmax.min = arr[low];
        return minmax;
    }

    if (high == low + 1) {
        if (arr[low] > arr[high]) {
            minmax.max = arr[low];
            minmax.min = arr[high];
        } else {
```

```
            minmax.max = arr[high];
            minmax.min = arr[low];
        }
        return minmax;
    }

    mid = (low + high) / 2;
    left = getMinMax(arr, low, mid);
    right = getMinMax(arr, mid + 1, high);

    minmax.max = (left.max > right.max) ? left.max : right.max;
    minmax.min = (left.min < right.min) ? left.min : right.min;

    return minmax;
}

int main() {
    int arr[100], n, i;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter the elements:\n");
    for (i = 0; i < n; i++) scanf("%d", &arr[i]);

    struct pair result = getMinMax(arr, 0, n - 1);

    printf("Minimum element: %d\n", result.min);
    printf("Maximum element: %d\n", result.max);

    return 0;
}
```

Algorithm:

- **Step 1:** Start the program.
- **Step 2:** Divide the array by calculating the mid index: $mid = l + (r - l) / 2$.
- **Step 3:** Recursively find the maximum and minimum of the left part by calling the same function.
- **Step 4:** Recursively find the maximum and minimum for the right part.
- **Step 5:** Finally, get the overall maximum and minimum by comparing the results of both halves.
- **Step 6:** Stop the program.

Child-Level Algorithm:

- **Step 1:** Imagine you have a big pile of numbered blocks and you want to find the biggest and smallest one.
- **Step 2:** Instead of looking at them all at once, split the pile into two smaller groups.
- **Step 3:** Ask a friend to find the biggest and smallest in the first group, while you do the same for the second group.
- **Step 4:** Once you both finish, compare your results.
- **Step 5:** The biggest of your two "big" blocks is the winner, and the smallest of your two "small" blocks is the tiny winner!

Output:

```
Enter number of elements: 3
Enter the elements:
0

7
6
Minimum element: 0
Maximum element: 7
```

Result: Thus, the C program for finding maximum and minimum numbers using the divide and conquer technique was executed successfully and the output was verified.

12.MERGE SORT AND QUICK SORT TIME ANALYSIS

Aim: To write a C program to implement Merge sort and Quick sort methods to sort an array, determine the time required to sort, repeat the experiment for different values of n, and plot a graph of the time taken versus n for both methods.

Simplified Code:

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <graphics.h>

void merge(int arr[], int l, int m, int r) {
    int i, j, k, n1 = m - l + 1, n2 = r - m;
    int *L = malloc(n1 * sizeof(int)), *R = malloc(n2 * sizeof(int));
    for (i = 0; i < n1; i++) L[i] = arr[l + i];
    for (j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    i = 0; j = 0; k = l;
    while (i < n1 && j < n2) arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
    free(L); free(R);
}

void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
```

```
        merge(arr, l, m, r);
    }
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high], i = (low - 1);
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
        }
    }
    int t = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = t;
    return (i + 1);
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n_list[] = {5000, 10000, 15000, 20000};
    double m_times[4], q_times[4];
    int gd = DETECT, gm, k, i;

    for (k = 0; k < 4; k++) {
```

```
    int n = n_list[k];
    int *a1 = malloc(n * sizeof(int)), *a2 = malloc(n * sizeof(int));
    for (i = 0; i < n; i++) a1[i] = a2[i] = rand() % 1000;

    clock_t s1 = clock(); mergeSort(a1, 0, n - 1); clock_t e1 = clock();
    m_times[k] = (double)(e1 - s1) / CLOCKS_PER_SEC;

    clock_t s2 = clock(); quickSort(a2, 0, n - 1); clock_t e2 = clock();
    q_times[k] = (double)(e2 - s2) / CLOCKS_PER_SEC;

    free(a1); free(a2);
}

initgraph(&gd, &gm, "");
line(50, 400, 500, 400); line(50, 400, 50, 50);
outtextxy(50, 30, "Green: Merge Sort | Red: Quick Sort");

for (k = 0; k < 4; k++) {
    int px = 50 + (k + 1) * 100;
    int py_m = 400 - (int)(m_times[k] * 10000);
    int py_q = 400 - (int)(q_times[k] * 10000);

    setcolor(GREEN); circle(px, py_m, 3);
    setcolor(RED); circle(px, py_q, 3);
    if (k > 0) {
        setcolor(GREEN); line(50 + k * 100, 400 - (int)(m_times[k-1] * 10000), px, py_m);
        setcolor(RED); line(50 + k * 100, 400 - (int)(q_times[k-1] * 10000), px, py_q);
    }
}
getch(); closegraph(); return 0;
```

}

Algorithm Breakdown:

- **Merge Sort:**
  - Start by declaring the array and variables.
  - Perform the merge function.
  - If left index is greater than right, return.
  - Calculate the midpoint, recursively call `mergesort` for both halves, and then merge them.
- **Quick Sort:**
  - Create a recursive function to implement the sort.
  - Partition the range and return the correct position of the pivot (pi).
  - Select the rightmost value as the pivot, compare elements, and perform the partition.
  - Recursively call `quicksort` for the parts to the left and right of pi.

Child-Level Algorithm:

- **Merge Sort:** Imagine you have a giant stack of messy papers. You split the stack in half and give each half to a friend to sort. Once they finish, you take their two sorted stacks and merge them together by looking at the top paper of each pile and picking the smaller one.
- **Quick Sort:** Imagine you pick one random student in a classroom to be the "Leader" (Pivot). You tell everyone shorter than the Leader to stand on the left and everyone taller to stand on the right. Now you have two smaller groups. You pick a new Leader for each group and repeat until everyone is in a perfect line from shortest to tallest.

Result: Thus, the C program for Merge sort and Quick sort methods was executed successfully and the output was verified.

13.N-QUEENS PROBLEM USING BACKTRACKING

Aim: To write a C program to implement the N-Queens problem using the backtracking technique.

Simplified Code:

C

```c
#include <stdio.h>
#include <stdbool.h>

int board[20], n;

void printSolution() {
    int i, j;
    printf("\nSolution:\n");
    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++) {
            if (board[i] == j) printf(" Q ");
            else printf(" . ");
        }
        printf("\n");
    }
}

bool isSafe(int row, int col) {
    for (int i = 1; i < row; i++) {
        if (board[i] == col || abs(board[i] - col) == abs(i - row))
            return false;
    }
    return true;
```

```
}

void solve(int row) {
    for (int col = 1; col <= n; col++) {
        if (isSafe(row, col)) {
            board[row] = col;
            if (row == n) printSolution();
            else solve(row + 1);
        }
    }
}

int main() {
    printf("Enter number of Queens: ");
    scanf("%d", &n);
    solve(1);
    return 0;
}
```

Output:

```
Plaintext

Enter number of Queens: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .
```

Solution:

. . Q .

Q . . .

. . . Q

. Q . .

Algorithm:

- **Step 1:** Initialize an empty chessboard of size $N \times N$.
- **Step 2:** Start with the leftmost column and place a queen in the first row.
- **Step 3:** Move to the next column and attempt to place a queen in a row that does not violate rules.
- **Step 4:** Repeat until all N queens are placed or it is impossible to place a queen in the current column.
- **Step 5:** If all N queens are placed, print the solution.
- **Step 6:** If a placement is impossible, backtrack to the previous column.
- **Step 7:** Remove the queen from the previous column and move it down one row.
- **Step 8:** Repeat until all possible configurations are tried.

Child-Level Algorithm:

- **Step 1:** Imagine you have 4 angry Queens and a 4x4 grid of squares.
- **Step 2:** The Queens are angry because if they are in the same row, column, or diagonal, they will fight!
- **Step 3:** Put the first Queen in the first row.
- **Step 4:** Try to put the second Queen in the second row where she can't "see" the first one.
- **Step 5:** If you get stuck and can't place a Queen, pick up the last Queen you placed and move her to a different spot.
- **Step 6:** Keep trying and moving them (backtracking) until everyone is sitting peacefully without fighting!

Result: Thus, the C program for the N-Queens problem was executed successfully and the output was verified.

14.TRAVELING SALESPERSON PROBLEM: OPTIMAL VS. APPROXIMATION

Aim: To write a C program to implement a scheme to find the optimal solution for the Traveling Salesperson Problem (TSP) and solve the same instance using an approximation algorithm to determine the error.

Simplified Code:

C

```c
#include <stdio.h>


int n, dist[10][10], visited[10];
int optimal_cost = 999;


void find_optimal(int curr, int count, int cost, int start) {
    if (count == n && dist[curr][start]) {
        if (cost + dist[curr][start] < optimal_cost)
            optimal_cost = cost + dist[curr][start];
        return;
    }
    for (int i = 1; i <= n; i++) {
        if (!visited[i] && dist[curr][i]) {
            visited[i] = 1;
            find_optimal(i, count + 1, cost + dist[curr][i], start);
            visited[i] = 0;
        }
    }
}


int main() {
```

```
int i, j, approx_cost = 0, curr = 1;
printf("Enter number of cities: ");
scanf("%d", &n);
printf("Enter distance matrix:\n");
for (i = 1; i <= n; i++)
    for (j = 1; j <= n; j++)
        scanf("%d", &dist[i][j]);

visited[1] = 1;
find_optimal(1, 1, 0, 1);

for (i = 1; i <= n; i++) visited[i] = 0;
visited[1] = 1;
for (i = 1; i < n; i++) {
    int next = -1, min = 999;
    for (j = 1; j <= n; j++) {
        if (!visited[j] && dist[curr][j] < min) {
            min = dist[curr][j];
            next = j;
        }
    }
    approx_cost += min;
    visited[next] = 1;
    curr = next;
}
approx_cost += dist[curr][1];

printf("\nOptimal Cost: %d", optimal_cost);
printf("\nApproximation Cost (Nearest Neighbor): %d", approx_cost);
printf("\nError: %d", approx_cost - optimal_cost);
```

```
    return 0;
}
```

Algorithm:

- **Step 1:** Start on an arbitrary vertex as the current vertex.
- **Step 2:** To find the optimal solution, use backtracking to check every possible path and record the lowest cost.
- **Step 3:** For the approximation, find the shortest edge connecting the current vertex and an unvisited vertex V.
- **Step 4:** Set the current vertex to V and mark V as visited.
- **Step 5:** If all vertices are visited, return to the start and terminate.
- **Step 6:** Calculate the error by subtracting the optimal cost from the approximate cost.

Child-Level Algorithm:

- **Step 1:** Imagine you are a delivery driver who needs to visit 4 houses and return home using the least amount of gas.
- **Step 2: The Perfect Way (Optimal):** Sit down with a map and calculate every single possible route you could take. Pick the shortest one.
- **Step 3: The Quick Way (Approximation):** Just look at where you are and drive to the house that is closest to you right now. Keep doing that until you've visited everyone.
- **Step 4:** The "Quick Way" is much faster to plan, but it might use a little more gas.
- **Step 5:** Subtract the "Perfect" gas amount from the "Quick" gas amount to see how much extra you used. That extra is your "Error."

Output:

```
Enter number of cities: 3
Enter distance matrix:
0 9 8
7 8 6
4 5 6

Optimal Cost: 19
Approximation Cost (Nearest Neighbor): 20
ERROR!
Error: 1
```

Result: Thus, the C program for finding the optimal and approximate solutions for the Traveling Salesperson Problem was executed successfully and the output was verified.

15.RANDOMIZED SELECT (Kth SMALLEST NUMBER)

Aim: To write a C program to implement a randomized algorithm to find the k^{th} smallest number in a given list.

Simplified Code:

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

void swap(int* a, int* b) {
    int t = *a; *a = *b; *b = t;
}

int partition(int arr[], int l, int r) {
    int x = arr[r], i = l;
    for (int j = l; j <= r - 1; j++) {
        if (arr[j] <= x) {
            swap(&arr[i], &arr[j]);
            i++;
        }
    }
    swap(&arr[i], &arr[r]);
    return i;
}

int randomPartition(int arr[], int l, int r) {
    int n = r - l + 1;
    int pivot = rand() % n;
    swap(&arr[l + pivot], &arr[r]);
    return partition(arr, l, r);
}

int kthSmallest(int arr[], int l, int r, int k) {
    if (k > 0 && k <= r - l + 1) {
        int pos = randomPartition(arr, l, r);
        if (pos - l == k - 1) return arr[pos];
        if (pos - l > k - 1) return kthSmallest(arr, l, pos - 1, k);
        return kthSmallest(arr, pos + 1, r, k - pos + l - 1);
    }
    return -1;
}
```

```
int main() {
    int arr[] = {12, 3, 5, 7, 4, 19, 26}, n = 7, k = 3;
    srand(time(NULL));
    printf("%d-th smallest element is %d", k, kthSmallest(arr, 0, n - 1, k));
    return 0;
}
```

Algorithm:

- **Step 1:** Find the k^{th} element in array A using a selection algorithm (Randomized Partition).
- **Step 2:** Initialize an empty list L and a counter to 0.
- **Step 3:** Compare elements against the pivot z.
- **Step 4:** If an element is less than z, increment the counter and add it to L.
- **Step 5:** If an element equals z and the count is less than k, add it to L and increment the counter.
- **Step 6:** Return the result.

Child-Level Algorithm:

- **Step 1:** Imagine you have a big bag of marbles of different sizes and you want the 3rd smallest one.
- **Step 2:** Reach in and grab a random marble to be your "Judge."
- **Step 3:** Put all marbles smaller than the Judge on the left and bigger ones on the right.
- **Step 4:** Count how many are on the left.
- **Step 5:** If there are exactly 2 marbles on the left, then your Judge is the 3rd smallest!
- **Step 6:** If there are too many on the left, throw away the big ones and repeat the game with the small ones.
- **Step 7:** If there are too few, look in the big pile for the marble you need.

Output;

```
3-th smallest element is 5

=== Code Execution Successful ===
```

Result: Thus, the C program to implement randomized algorithms for finding the k^{th} smallest number was executed successfully and the output was verified.